LangGraph Tutorial

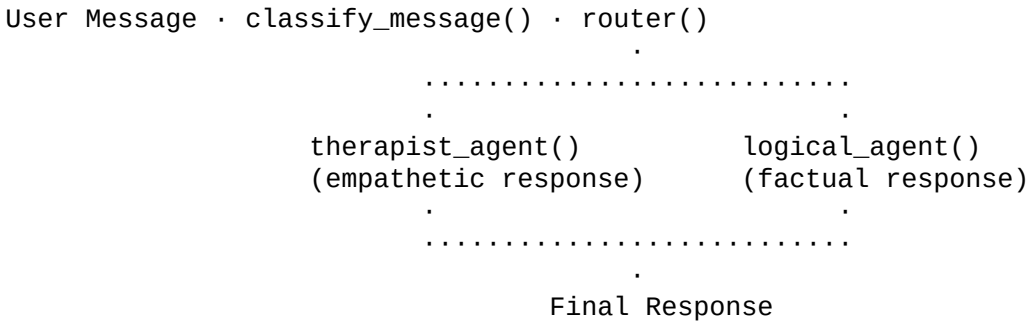
A **dual-agent message router** built with **LangGraph** that classifies messages as emotional or logical and routes them to specialized response agents.

· Features

- ·**Message Classification** · Automatically detect emotional vs logical intent
- ·**Therapist Agent** · Empathetic responses for emotional messages
- ·**Logical Agent** · Factual, direct responses for logical queries
- ·**Conditional Routing** · LangGraph state machine with dynamic branching
- ·**Structured Output** · Pydantic-based classification schema

---

·· Architecture



· Getting Started

```
cd LangGraph-Tutorial
uv sync
uv run python main.py
```

Environment Variables (.env)

```
ANTHROPIC_API_KEY=...
```

· Logic Flow

1. **Input** · User sends a message
2. **Classification** ·`classify_message()` uses Claude 3.5 Sonnet with structured output (`MessageClassifier` Pydantic model)
3. **Routing** ·`router()` reads classification and routes to appropriate agent
4. **Response Generation**:
   - *Emotional* ·`therapist_agent()` generates empathetic, supportive response
   - *Logical* ·`logical_agent()` generates factual, direct response

5 **Output** · Response displayed to user

Key Components

| Component | Type | Purpose | |-----------|------|---------| | `MessageClassifier` | Pydantic Model | Structured classification output | | `State` | TypedDict | LangGraph state container | | `classify_message()` | Node | LLM-based message classification | | `router()` | Conditional Edge | Routes based on classification | | `therapist_agent()` | Node | Empathetic response generator | | `logical_agent()` | Node | Factual response generator |

· Dependencies

`langchain`, `langgraph`, `langchain-anthropic`, `python-dotenv`

· License